STAR RX72N - rx_bus Library

1.0.0

Generated by Doxygen 1.16.1

Chapter 1

Test List

Global internal_add_bus_locked (rx_bus_manager_t *manager, rx_bus_config_t *bus_config)

- test_rx_bus_manager.c::test_add_bus_success()
- test_rx_bus_manager.c::test_add_bus_duplicate_name()
- test_rx_bus_manager.c::test_add_bus_max_capacity()

Global internal_validate_bus_manager_interfaces (rx_error_interface_t *error_iface, rx_pin_interface_t *pin_iface)

- test_rx_bus_manager.c::test_bus_manager_init_success()
- test_rx_bus_manager.c::test_bus_manager_init_null_params()
- test_rx_bus_manager.c::test_bus_manager_init_invalid_interfaces()

Global rx_bus_config_init_adc (rx_bus_config_t *config, const char *name, const uint8_t unit, const uint8_t channel, const uint8_t bits)

- test_rx_bus_config.c::test_init_adc_success()
- test_rx_bus_config.c::test_init_adc_null_params()
- test_rx_bus_config.c::test_init_adc_invalid_unit()
- test_rx_bus_config.c::test_init_adc_invalid_channel()
- test_rx_bus_config.c::test_init_adc_invalid_resolution()

Global rx_bus_config_init_gpio (rx_bus_config_t *config, const char *name, rx_port_pin_t pin)

- test_rx_bus_config.c::test_init_gpio_success()
- test_rx_bus_config.c::test_init_gpio_null_params()
- test_rx_bus_config.c::test_init_gpio_invalid_pin()

Global rx_bus_config_init_i2c (rx_bus_config_t *config, const char *name, const uint8_t channel, const uint8_t device_addr, const rx_port_pin_t sda_pin, const rx_port_pin_t scl_pin, const uint32_t frequency_hz)

- test_rx_bus_config.c::test_init_i2c_success()
- test_rx_bus_config.c::test_init_i2c_null_params()
- test_rx_bus_config.c::test_init_i2c_invalid_pins()
- test_rx_bus_config.c::test_init_i2c_invalid_channel()
- test_rx_bus_config.c::test_init_i2c_invalid_address()

Global rx_bus_config_init_onewire (rx_bus_config_t *config, const char *name, rx_port_pin_t pin)

test_rx_bus_config.c::test_init_onewire_success()

test_rx_bus_config.c::test_init_onewire_null_params()

test_rx_bus_config.c::test_init_onewire_invalid_pin()

Global rx_bus_config_init_uart (rx_bus_config_t *config, const char *name, const uint8_t channel, const rx_port_pin_t tx_pin, const rx_port_pin_t rx_pin, const uint32_t baudrate)

test_rx_bus_config.c::test_init_uart_success()

test_rx_bus_config.c::test_init_uart_null_params()

test_rx_bus_config.c::test_init_uart_invalid_channel()

test_rx_bus_config.c::test_init_uart_invalid_pins()

test_rx_bus_config.c::test_init_uart_zero_baudrate()

Global rx_bus_i2c_init (rx_bus_manager_t *manager, const char *bus_name)

test_rx_bus_i2c.c::test_i2c_init_success()

test_rx_bus_i2c.c::test_i2c_init_null_params()

test_rx_bus_i2c.c::test_i2c_init_not_found()

test_rx_bus_i2c.c::test_i2c_init_wrong_type()

Global rx_bus_i2c_read (rx_bus_manager_t *manager, const char *bus_name, uint8_t *data, uint16_t length)

test_rx_bus_i2c.c::test_i2c_read_success()

Global rx_bus_i2c_write (rx_bus_manager_t *manager, const char *bus_name, const uint8_t *data, const uint16_t length)

test_rx_bus_i2c.c::test_i2c_write_success()

test_rx_bus_i2c.c::test_i2c_write_null_params()

test_rx_bus_i2c.c::test_i2c_write_not_initialized()

test_rx_bus_i2c.c::test_i2c_write_nack()

Global rx_bus_i2c_write_read (rx_bus_manager_t *manager, const char *bus_name, const uint8_t *write_data, const uint16_t write_length, uint8_t *read_data, const uint16_t read_length)

test_rx_bus_i2c.c::test_i2c_write_read_success()

test_rx_bus_i2c.c::test_i2c_write_read_multi_byte()

test_rx_bus_i2c.c::test_i2c_write_read_null_params()

test_rx_bus_i2c.c::test_i2c_write_read_not_initialized()

test_rx_bus_i2c.c::test_i2c_write_read_invalid_register()

Global rx_bus_manager_deinit (rx_bus_manager_t *manager)

test_rx_bus_manager.c::test_bus_manager_deinit_success()

test_rx_bus_manager.c::test_bus_manager_deinit_null_ptr()

test_rx_bus_manager.c::test_bus_manager_deinit_with_buses()

Global rx_bus_manager_remove_bus (rx_bus_manager_t *manager, const char *name)

test_rx_bus_manager.c::test_remove_bus_success()

test_rx_bus_manager.c::test_remove_bus_not_found()

test_rx_bus_manager.c::test_remove_bus_head()

test_rx_bus_manager.c::test_remove_bus_tail()

Chapter 2

Directory Hierarchy

2.1 Directories

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

Chapter 4

File Index

4.1 File List

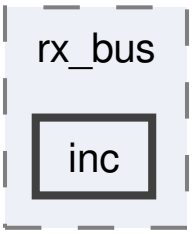
Here is a list of all files with brief descriptions:

Chapter 5

Directory Documentation

5.1 libs/rx_bus/inc Directory Reference

Directory dependency graph for inc:

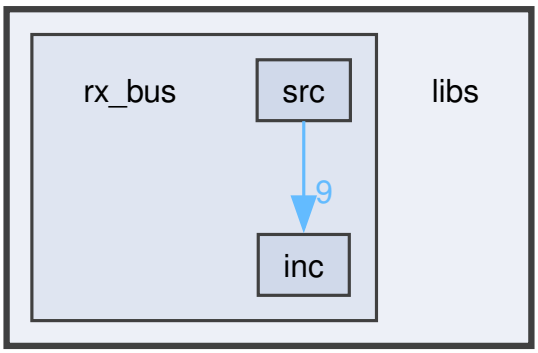


Files

- file rx_bus_adc.h
  ADC (Analog-to-Digital Converter) Bus Abstraction for RX72N.
- file rx_bus_command.h
  Command Pattern interface for polymorphic bus operations.
- file rx_bus_config.h
  Bus configuration creation helpers for RX72N peripheral abstraction.
- file rx_bus_gpio.h
  GPIO (General Purpose Input/Output) Bus Abstraction for RX72N.
- file rx_bus_i2c.h
  I2C (Inter-Integrated Circuit) Bus Abstraction for RX72N.
- file rx_bus_manager.h
  Unified Bus Manager API for RX72N Multi-Protocol Communication.
- file rx_bus_onewire.h
  OneWire (1-Wire) Bus Abstraction for RX72N.
- file rx_bus_types.h
  Bus Manager Type Definitions for RX72N Multi-Protocol Communication.

5.2 libs Directory Reference

Directory dependency graph for libs:

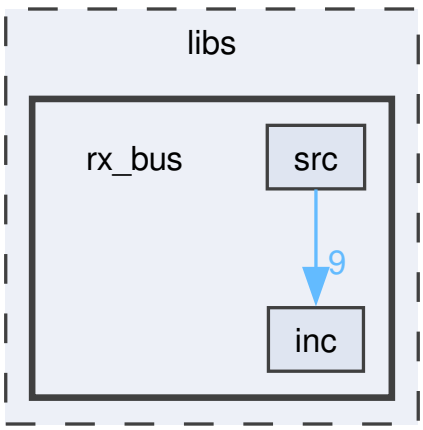


Directories

- directory rx_bus

5.3 libs/rx_bus Directory Reference

Directory dependency graph for rx_bus:

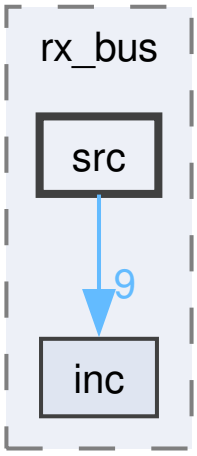


Directories

- directory inc
- directory src

5.4 libs/rx_bus/src Directory Reference

Directory dependency graph for src:



Files

- file rx_bus_adc.c
  ADC (Analog-to-Digital Converter) Bus Abstraction Implementation for RX72N.
- file rx_bus_config.c
  Bus Configuration Creation Helpers - Static Initialization Pattern.
- file rx_bus_gpio.c
  GPIO Bus Abstraction Implementation for Renesas RX72N.
- file rx_bus_i2c.c
  I2C Bus Abstraction Implementation - Callback-Based Thread-Safe Operations.
- file rx_bus_manager.c
  Bus Manager Implementation - Thread-Safe Multi-Protocol Registry.
- file rx_bus_onewire.c
  OneWire (1-Wire) Bus Implementation Using GPIO Bit-Banging.

Chapter 6

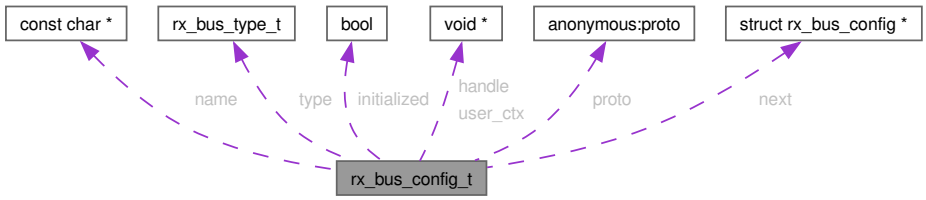
Data Structure Documentation

6.1 rx_bus_config_t Struct Reference

Bus configuration structure (linked list node).

#include <rx_bus_types.h>

Collaboration diagram for rx_bus_config_t:



Data Fields

- const char * name
  Unique bus name for lookup.
- rx_bus_type_t type
  Bus protocol type.
- bool initialized
  Bus initialization status.
- void * handle
  Driver-specific handle (opaque pointer).
- void * user_ctx
  User context for callbacks.
- union {
  rx_gpio_bus_config_t gpio
  rx_adc_bus_config_t adc
  rx_i2c_bus_config_t i2c
  rx_spi_bus_config_t spi
  rx_uart_bus_config_t uart
  rx_onewire_bus_config_t onewire
  } proto

  Protocol-specific configuration union.
- struct rx_bus_config * next
  Next bus in linked list.

6.1.1 Detailed Description

Bus configuration structure (linked list node).

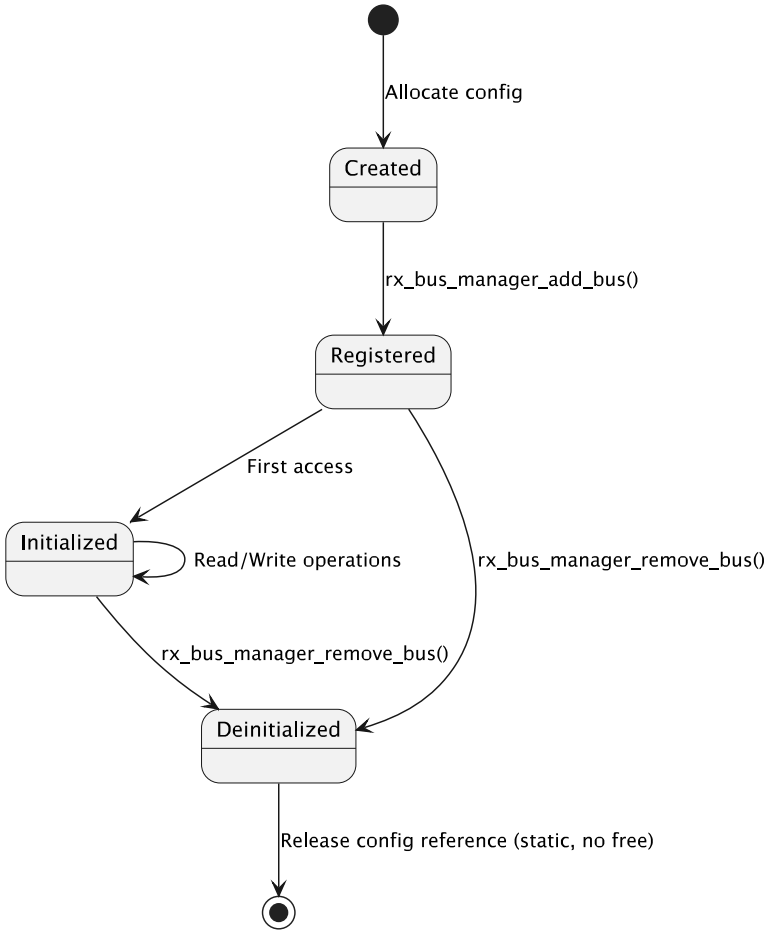
Contains complete configuration for a specific bus instance. Each bus has:

- Unique name for lookup
- Protocol type and protocol-specific settings
- Runtime state (initialized flag, driver handle)
- User context for callbacks
- Link to next bus in manager's linked list

6.1.1.1 Memory Layout

Offset	Size	Field	Type	Notes
0	4	name	const char*	Pointer to name string
4	1	type	rx_bus_type_t	Bus protocol type
5	1	initialized	bool	Runtime state flag
6	2	(padding)	-	Alignment padding
8	4	handle	void*	Driver-specific handle
12	4	user_ctx	void*	User callback context
16	24	proto	union	Protocol-specific config
40	4	next	struct*	Linked list pointer
Total	44	-	-	Actual may vary with alignment

6.1.1.2 Bus Lifecycle State Machine



6.1.1.3 Field Constraints

Field	Constraint	Validation
name	Non-NULL, <=31 chars	rx_bus_manager_add_bus()
type	0 <= type < k_bus_type_max	rx_bus_manager_add_bus()
proto.*	Type-specific	Protocol validators

Usage Example - SPI Bus Configuration:

```
// Statically allocate SPI bus config for RPi5 communication (zero dynamic allocation)
static rx_bus_config_t s_rpi5_spi = {
    .name = "rpi5_spi",
    .type = k_bus_type_spi,
};

s_rpi5_spi.proto.spi = (rx_spi_bus_config_t){
    .channel = k_rspi_channel_0,
    .copi_pin = k_port_pin_p26,  // RSPI0 COPI
    .cipo_pin = k_port_pin_p30,  // RSPI0 CIPO
    .sck_pin = k_port_pin_p27,   // RSPI0 SCK
    .cs_pin = k_port_pin_p54,    // GPIO chip select
    .frequency_hz = k_rspi_freq_10mhz,
    .mode = k_rspi_mode_0        // CPOL=0, CPHA=0
};

// Register with bus manager
rx_err_t err = rx_bus_manager_add_bus(&manager, &s_rpi5_spi);
```

Usage Example - I2C Bus Configuration:

```
// Statically allocate I2C bus config for IMU sensor (zero dynamic allocation)
static rx_bus_config_t s_imu_i2c = {
    .name = "imu",
    .type = k_bus_type_i2c,
};

s_imu_i2c.proto.i2c = (rx_i2c_bus_config_t){
    .channel = k_riic_channel_0,
    .sda_pin = k_port_pin_p16,
    .scl_pin = k_port_pin_p15,
    .frequency_hz = k_riic_freq_400khz,  // Fast mode
    .device_addr = k_mpu6050_i2c_addr     // MPU6050 7-bit address
};

rx_bus_manager_add_bus(&manager, &s_imu_i2c);
```

Invariant

name must be unique within bus manager

type must be valid (< k_bus_type_max)

proto union interpretation depends on type field

Note

Bus manager holds a reference to config; caller owns statically-allocated config for its entire lifetime

Warning

Do not modify config after registration without mutex

See also

rx_bus_manager_add_bus() Register bus with manager

rx_bus_manager_find_bus() Lookup bus by name

rx_bus_type_t Bus type enumeration

Since

Version 1.0.0

Definition at line 579 of file rx_bus_types.h.

6.1.2 Field Documentation

6.1.2.1 handle

void* rx_bus_config_t::handle

Driver-specific handle (opaque pointer).

Holds driver state after initialization. Contents depend on bus type:

- GPIO: NULL (stateless)
- I2C: RIIC state structure
- SPI: RSPI state structure
- ADC: ADC unit state

Default: NULL (set by memset during allocation)

Note

Managed by driver, do not modify directly

Definition at line 624 of file rx_bus_types.h.

Referenced by internal_acquire_state(), internal_get_state(), internal_release_state(), and internal_set_common_field

6.1.2.2 initialized

bool rx_bus_config_t::initialized

Bus initialization status.

Runtime state tracking. Set true after successful hardware init.

- false: Config registered but hardware not initialized
- true: Hardware initialized and ready for operations

Default: false (set by memset during allocation)

Note

Set by bus manager during first access, not by user

Definition at line 611 of file rx_bus_types.h.

Referenced by internal_adc_init_callback(), internal_adc_read_callback(), internal_adc_voltage_callback(), internal_gpio_init_callback(), internal_gpio_read_callback(), internal_gpio_toggle_callback(), internal_gpio_write_callback(), internal_i2c_handle_shared_channel(), internal_i2c_init_callback(), internal_i2c_read_callback(), internal_i2c_write_callback(), internal_i2c_write_read_callback(), internal_onewire_deinit_callback(), internal_onewire_init_callback(), internal_onewire_match_rom_callback(), internal_onewire_read_bit_callback(), internal_onewire_read_buffer_callback(), internal_onewire_read_byte_callba internal_onewire_read_rom_callback(), internal_onewire_skip_rom_callback(), internal_onewire_write_bit_callback internal_onewire_write_buffer_callback(), internal_onewire_write_byte_callback(), internal_set_common_fields(), and internal_validate_search_params().

6.1.2.3 name

const char* rx_bus_config_t::name

Unique bus name for lookup.

Human-readable identifier for the bus instance. Used by rx_bus_manager_find_bus() and logging.

Valid Range: Non-NULL, <=31 characters (k_max_bus_name_len)

Examples: "rpi5_spi", "imu", "temp_sensor", "debug_uart"

Invariant

Must be unique within bus manager

Warning

Must remain valid for lifetime of bus config

Definition at line 590 of file rx_bus_types.h.

Referenced by internal_add_bus_locked(), internal_set_common_fields(), rx_bus_manager_add_bus(), rx_bus_manager_find_bus(), and rx_bus_manager_with_bus().

6.1.2.4 next

struct rx_bus_config* rx_bus_config_t::next

Next bus in linked list.

Points to next bus config in manager's internal list. NULL if this is the last (or only) bus.

Default: NULL (set by memset during allocation)

Note

Managed by bus manager, do not modify directly

Definition at line 671 of file rx_bus_types.h.

Referenced by internal_add_bus_locked(), internal_set_common_fields(), rx_bus_manager_deinit(), rx_bus_manager_find_bus(), rx_bus_manager_remove_bus(), and rx_bus_manager_with_bus().

6.1.2.5 [union]

union { ... } rx_bus_config_t::proto

Protocol-specific configuration union.

Contains configuration data specific to the bus protocol. The active member is determined by the type field.

type Value	Active Member	Purpose
k_bus_type_gpio	proto.gpio	GPIO pin config
k_bus_type_adc	proto.adc	ADC channel config
k_bus_type_i2c	proto.i2c	I2C bus config
k_bus_type_spi	proto.spi	SPI bus config
k_bus_type_uart	proto.uart	UART config
k_bus_type_onewire	proto.onewire	1-Wire config

Warning

Reading wrong union member causes undefined behavior

Invariant

Must interpret based on type field

Referenced by internal_adc_init_callback(), internal_adc_read_callback(), internal_adc_voltage_callback(), internal_drive_low(), internal_gpio_init_callback(), internal_gpio_read_callback(), internal_gpio_toggle_callback(), internal_gpio_write_callback(), internal_i2c_handle_shared_channel(), internal_i2c_init_callback(), internal_i2c_read_callback(), internal_i2c_write_callback(), internal_i2c_write_read_callback(), internal_onewire_init_callback(), internal_read_line(), internal_set_drive_mode(), rx_bus_config_init_adc(), rx_bus_config_init_gpio(), rx_bus_config_init_i2c(), rx_bus_config_init_onewire(), and rx_bus_config_init_uart().

6.1.2.6 type

rx_bus_type_t rx_bus_config_t::type

Bus protocol type.

Determines which member of proto union to use and which driver functions to invoke for operations.

Valid Range: 0 <= type < k_bus_type_max

See also

rx_bus_type_t Protocol type enumeration

Definition at line 600 of file rx_bus_types.h.

Referenced by internal_adc_init_callback(), internal_gpio_init_callback(), internal_i2c_init_callback(), internal_onewire_match_rom_callback(), internal_onewire_read_bit_callback(), internal_onewire_read_buffer_callb internal_onewire_read_byte_callback(), internal_onewire_read_rom_callback(), internal_onewire_skip_rom_callbac internal_onewire_write_bit_callback(), internal_onewire_write_buffer_callback(), internal_onewire_write_byte_call internal_set_common_fields(), and internal_validate_search_params().

6.1.2.7 user_ctx

void* rx_bus_config_t::user_ctx

User context for callbacks.

Opaque pointer passed to user callbacks during bus operations. Allows user to associate application-specific data with bus.

Default: NULL (set by memset during allocation)

Note

Bus manager does not dereference this pointer

Definition at line 634 of file rx_bus_types.h.

Referenced by internal_set_common_fields().

The documentation for this struct was generated from the following file:

- libs/rx_bus/inc/rx_bus_types.h

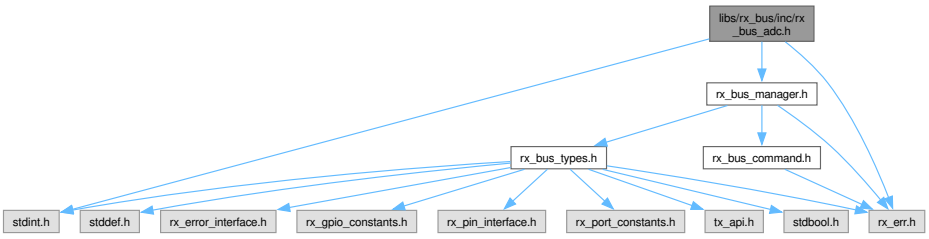
Chapter 7

File Documentation

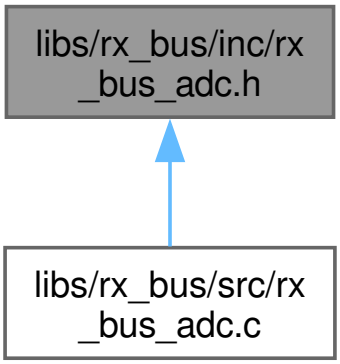
7.1 libs/rx_bus/inc/rx_bus_adc.h File Reference

ADC (Analog-to-Digital Converter) Bus Abstraction for RX72N.

#include <stdint.h>
#include "rx_bus_manager.h"
#include "rx_err.h"
Include dependency graph for rx_bus_adc.h:



This graph shows which files directly or indirectly include this file:



Functions

- rx_err_t rx_bus_adc_init (rx_bus_manager_t *manager, const char *bus_name)

  Initialize ADC channel through bus manager.
- rx_err_t rx_bus_adc_read (rx_bus_manager_t *manager, const char *bus_name, uint16_t *value)

  Read ADC raw value through bus manager.
- rx_err_t rx_bus_adc_read_voltage_mv (rx_bus_manager_t *manager, const char *bus_name, uint32_t *voltage_mv)

  Read ADC voltage in millivolts through bus manager.

7.1.1 Detailed Description

ADC (Analog-to-Digital Converter) Bus Abstraction for RX72N.

Provides bus manager integration for Renesas S12ADFa (Successive Approximation ADC Fast) peripheral operations. Wraps the low-level ADC HAL with the bus abstraction pattern for consistent API across all bus types (I2C, SPI, UART, GPIO, 1-Wire, ADC) with mutex-protected access.

ADC Overview

The S12ADFa (Successive Approximation ADC Fast) is a 12-bit ADC peripheral:

- 12-bit resolution: 0-4095 counts (4096 levels)
- Multiple units: S12AD0 and S12AD1 (28 total channels)
- Sampling time: 0.5-16.5 us configurable
- Conversion time: ∼1.5 us @ 12-bit resolution
- Input range: 0V to VREFH (typically 3.3V)
- Trigger modes: Software, hardware, periodic
- Averaging: 2, 4, 8, 16, or 32 samples

Conversion Process

```
* ADC Conversion Sequence (Software Trigger):
*
* 1. Sample Phase (ADCS = sampling time):
*    Input voltage charges internal sample & hold capacitor
*    Duration: 0.5 - 16.5 us (configurable)
*
* 2. Conversion Phase (12 clock cycles):
*    Successive approximation algorithm
*    Binary search from MSB to LSB
*    Duration: ~1.5 us @ 25 MHz PCLKD
*
* 3. Result Ready:
*    12-bit result available in ADDRn register
*    Conversion end interrupt (ADIE) if enabled
*
* Total Time = Sampling Time + Conversion Time
* Example: 3 us + 1.5 us = 4.5 us per channel
*
```

Resolution and Precision

Resolution	Counts	LSB @ 3.3V	LSB @ 5V	Use Case
12-bit	4096	0.805 mV	1.221 mV	Standard (default)
10-bit	1024	3.223 mV	4.883 mV	Faster sampling
8-bit	256	12.89 mV	19.53 mV	High-speed

ADC Formula

The ADC conversion follows this equation:

$$ADC_{raw} = \frac{V_{in}}{V_{REF}} \times (2^{bits} - 1)$$

Where:

- V_{in} = input voltage (0 to VREF)
- V_{REF} = reference voltage (typically 3.3V or 5V)
- $bits$ = resolution (8, 10, or 12)

Voltage conversion (reverse):

$$V_{in} = \frac{ADC_{raw} \times V_{REF}}{2^{bits} - 1}$$

Example for 12-bit @ 3.3V VREF:

$$V_{mV} = \frac{ADC_{raw} \times 3300}{4095}$$

Sampling Time Calculation

Sampling time depends on PCLKD and ADCS register:

$$T_{sample} = \frac{ADCS + 1}{PCLKD}$$

Where:

- ADCS = sampling state count (0-255)
- PCLKD = peripheral clock D (60 MHz on RX72N)

Example: ADCS = 74 @ 60 MHz:

$$T_{sample} = \frac{74 + 1}{60 \times 10^6} = 1.25\mu s$$

Performance Characteristics

Metric	Value	Notes
Init time	~50 us	One-time setup
Single conversion	4.5 us	3 us sample + 1.5 us conversion
Maximum rate	222 kSPS	@ minimum sampling time
Practical rate	100 kSPS	With adequate settling time
INL error	+/- 2 LSB	Integral non-linearity
DNL error	+/- 1 LSB	Differential non-linearity
Offset error	+/- 10 mV	Typical @ 25degC
Gain error	+/- 1%	Typical

Memory Usage

Component	Size	Description
Code (.text)	~900 bytes	All API functions
Stack	48 bytes	Maximum call depth
No heap	0	Static allocation only

Hardware Requirements

Requirement	Specification
VREFH	3.3V or 5V (reference voltage)
VREFL	0V (ground)
V_IN range	VREFL to VREFH
Source impedance	< 10 kOhm (for fast settling)
Bypass cap	0.1 uF ceramic on VREFH
Input cap	10 nF on each analog input

Channel Assignments (STAR Project - 144-pin LFQFP)

Unit	Channel	Pin	Signal	Range	Notes
S12AD0	AN007	P47	Motor 0 Current	0-3.3V	DRV8263H-Q1 IPROPI
S12AD0	AN006	P46	Motor 1 Current	0-3.3V	DRV8263H-Q1 IPROPI
S12AD0	AN005	P45	Motor 2 Current	0-3.3V	DRV8263H-Q1 IPROPI
S12AD0	AN004	P44	Motor 3 Current	0-3.3V	DRV8263H-Q1 IPROPI

Current Sensing (DRV8263H-Q1 IPROPI)

The DRV8263H-Q1 motor driver outputs a current proportional voltage on IPROPI:

- Current mirror: 202 uA/A (202 uA output per 1A load current)
- Sense resistor: 5100 Ohm (1% tolerance)
- Conversion: I_motor = V_IPROPI / 1.0302
- Derivation: V_IPROPI = I_motor * 202e-6 * 5100 = I_motor * 1.0302

Example: V_IPROPI = 1.65V (mid-scale):

$$I_{motor} = \frac{1.65}{1.0302} = 1.602A$$

Error Sources and Mitigation

Error Source	Magnitude	Mitigation
Quantization	+/- 0.5 LSB	Averaging (2-32 samples)
Noise	+/- 2 LSB	Low-pass filter on input
Offset drift	+/- 5 mV/degC	Temperature compensation
Gain error	+/- 1%	Calibration lookup table
Source impedance	Variable	Buffer amplifier if > 10 kOhm

Thread Safety

All functions are thread-safe when used with the bus manager:

- Bus manager provides mutex protection per ADC operation
- Timeout on mutex acquisition prevents deadlock
- Safe to call from multiple ThreadX threads
- Conversion started after mutex acquired (atomic)

Module Dependencies

- rx_bus_manager.h: Bus abstraction and mutex protection
- rx_err.h: Error code definitions
- rx_adc.h: Low-level S12ADFa hardware driver
- rx_mpc.h: Pin function selection (AN pins)

NASA Power of 10 Compliance

- Rule 1: [OK] No goto, setjmp, or recursion
- Rule 2: [OK] All loops bounded (conversion timeout)
- Rule 3: [OK] No dynamic memory allocation
- Rule 4: [OK] Functions under 60 lines
- Rule 5: [OK] Input validation via RX_CHECK_NULL_PTR
- Rule 6: [OK] Variables at smallest scope
- Rule 7: [OK] All return values checked
- Rule 8: [OK] Limited preprocessor (typed enums preferred)
- Rule 9: [OK] Function pointers only for bus abstraction (DIP)
- Rule 10: [OK] Compiled with -Wall -Wextra -Werror

SOLID Principles

- S: Single Responsibility - ADC analog input operations only
- O: Open/Closed - Extensible via bus manager config
- L: Liskov Substitution - Implements rx_bus_interface_t
- I: Interface Segregation - Minimal 3-function API
- D: Dependency Inversion - Depends on abstractions (bus manager)

Usage Example - Motor Current Monitoring

```
// Initialize bus manager
rx_bus_manager_t manager;
rx_bus_manager_init(&manager);

// Register ADC bus for motor 0 current sensing
rx_bus_config_t current_config = {
    .type = k_rx_bus_type_adc,
    .adc = {
        .unit = 0,          // S12AD0
        .channel = 0,       // AN000
        .resolution = 12,   // 12-bit
        .vref_mv = 3300,    // 3.3V reference
    }
};
```

```
rx_bus_register(&manager, "motor0_current", &current_config);

// Initialize ADC
rx_err_t err = rx_bus_adc_init(&manager, "motor0_current");
if (err != k_rx_ok) {
    return err;
}

// Read motor current
uint32_t voltage_mv = 0;
err = rx_bus_adc_read_voltage_mv(&manager, "motor0_current", &voltage_mv);
if (err == k_rx_ok) {
    // Convert IPROPI voltage to motor current (mA)
    // DRV8263H-Q1: V_IPROPI = I_motor * k_drv8263h_mirror_ua_per_a * 1e-6
    //              * k_drv8263h_sense_ohm = I_motor * 1.0302
    // I_motor = V_IPROPI / 1.0302
    // Fixed-point: scale = 10000, divisor = 10302 (1.0302 * 10000)
    enum : uint32_t {
        k_scale = 10000,
        k_divisor = 10302,
        k_overcurrent_ma = 5000
    };
    uint32_t current_ma = (voltage_mv * k_scale) / k_divisor;

    if (current_ma > k_overcurrent_ma) {
        // Overcurrent - disable motor
        motor_emergency_stop();
    }
}
```

Usage Example - ADC Voltage Monitoring

```
// Register ADC for power rail voltage (1:11 divider)
rx_bus_config_t power_rail_config = {
    .type = k_rx_bus_type_adc,
    .adc = {
        .unit = 0,
        .channel = 4,       // AN004
        .resolution = 12,
        .vref_mv = 3300,
    }
};
rx_bus_register(&manager, "power_rail", &power_rail_config);
rx_bus_adc_init(&manager, "power_rail");

// Read supply voltage
uint32_t adc_voltage_mv = 0;
err = rx_bus_adc_read_voltage_mv(&manager, "power_rail", &adc_voltage_mv);
if (err == k_rx_ok) {
    // Scale up by divider ratio (1:11)
    uint32_t supply_voltage_mv = adc_voltage_mv * 11;

    if (supply_voltage_mv < 11000) {  // 11V low supply warning
        rx_log_warn("POWER", "Low supply voltage: %d mV", supply_voltage_mv);
    }
}
```

Usage Example - Raw ADC Reading

```
// Read raw ADC counts (no voltage conversion)
uint16_t raw_value = 0;
err = rx_bus_adc_read(&manager, "sensor_input", &raw_value);
if (err == k_rx_ok) {
    // raw_value is 0-4095 for 12-bit ADC
    // Convert to voltage manually
    uint32_t voltage_mv = (raw_value * 3300UL) / 4095;

    // Or use for lookup table
    uint32_t temperature_c = ntc_lookup_table[raw_value];
}
```

See also

rx_bus_manager.h Bus manager API for registration and lookup

rx_bus_types.h Bus type definitions and configurations

rx_adc.h Low-level S12ADFa hardware driver

Version

1.0.0

Date

2026-01-29

Copyright

Definition in file rx_bus_adc.h.

7.1.2 Function Documentation

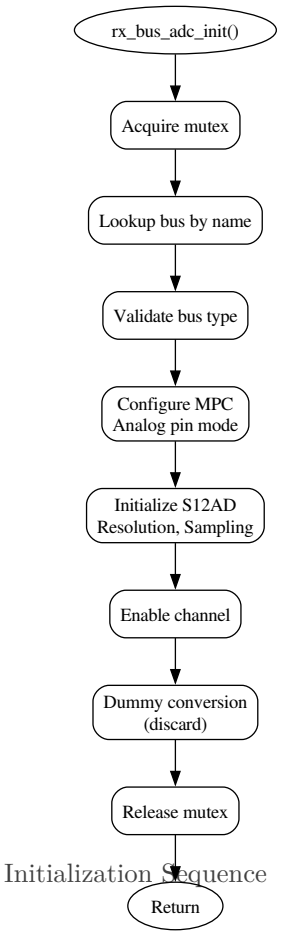
7.1.2.1 rx_bus_adc_init()

```
rx_err_t rx_bus_adc_init (
            rx_bus_manager_t * manager,
            const char * bus_name)    [nodiscard]
```

Initialize ADC channel through bus manager.

Configures the S12ADFa ADC peripheral for analog voltage measurement. This function must be called before any read operations. The initialization process:

1. Acquires bus manager mutex (thread-safe)
2. Looks up bus configuration by name
3. Validates bus type is ADC
4. Configures MPC for analog input pin function
5. Initializes S12AD unit with resolution and sampling time
6. Enables ADC channel
7. Performs initial conversion (discard for settling)
8. Releases mutex



Initialization Sequence

ADC Configuration:

The initialization configures:

- Resolution: 12-bit (4096 counts, 0.805 mV/LSB @ 3.3V)
- Sampling time: 3 us (adequate for most sensors)
- Conversion time: 1.5 us (12 clocks @ 25 MHz)
- Reference: VREFH pin (typically 3.3V or 5V)
- Trigger: Software (manual start per conversion)

Parameters

in	manager	Bus manager instance • Must be initialized via rx_bus_manager_init() • Must have ADC bus registered via rx_bus_register()
in	bus_name	ADC bus name • Null-terminated string • Must match name used in rx_bus_register() • Maximum 31 characters

Returns

rx_err_t Error code indicating result

Return values

k_rx_ok	Success, ADC channel initialized and ready
k_rx_err_null_ptr	nullptr in manager or bus_name
k_rx_err_not_found	Bus name not found in manager
k_rx_err_invalid_arg	Bus is not ADC type
k_rx_err_timeout	Mutex acquisition timeout (default 1000ms)
k_rx_err_hw_init_failed	ADC hardware initialization failed

Precondition

manager must be initialized via rx_bus_manager_init()

Bus must be registered via rx_bus_register() with type k_rx_bus_type_adc

Analog input pin must not be driven externally above VREFH

Postcondition

S12AD unit configured and enabled

Analog pin configured for ADC input

Bus state set to initialized

Initial dummy conversion performed (settling time)

Note

Thread-safe via bus manager mutex

Call only once per channel; subsequent calls return k_rx_err_invalid_state

First conversion after init should be discarded (settling time)

Warning

Do not exceed VREFH on analog input (damage possible)

Do not call from interrupt context (mutex wait)

Ensure source impedance $<$ 10 kOhm for accurate readings

Performance:

- Execution time: ~50 us typical (ADC setup + dummy conversion)
- Mutex timeout: 1000 ms (configurable via bus manager)

Example - Current Sensor Initialization:

```
rx_bus_config_t current_config = {
    .type = k_rx_bus_type_adc,
    .adc = {
        .unit = 0,
        .channel = 0,       // AN000 - motor current
        .resolution = 12,
        .vref_mv = 3300,
    }
};
rx_bus_register(&manager, "motor_current", &current_config);

rx_err_t err = rx_bus_adc_init(&manager, "motor_current");
if (err != k_rx_ok) {
    rx_log_error("ADC", "Failed to init current sensor");
    return err;
}
```

See also

rx_bus_manager_init() Initialize bus manager first

rx_bus_register() Register bus before initialization

rx_bus_adc_read() Read raw ADC value

rx_bus_adc_read_voltage_mv() Read voltage in millivolts

Since

Version 1.0.0

Public API function to initialize an S12ADFa ADC channel for analog voltage measurement. Delegates to bus manager which calls internal_adc_init_callback() with mutex protection and bus lookup.

Implementation pattern:

1. Validate input pointers (manager, bus_name)
2. Create stack-allocated context for operation result
3. Call rx_bus_manager_with_bus() to execute callback
4. Extract result from context
5. Return result to caller

This function demonstrates the callback pattern for bus operations:

- Prepares context on stack (zero allocation)
- Delegates to bus manager for mutex protection
- Callback executes with mutex held
- Results extracted after mutex released

Precondition

manager must be initialized

Bus must be registered with type k_rx_bus_type_adc

Analog input voltage must be within 0 to VREFH

Postcondition

S12AD unit configured and enabled

Analog pin configured for ADC input

Bus marked as initialized

Initial dummy conversion performed (settling)

Note

Thread-safe via bus manager mutex

Stack usage: 4 bytes for context

Execution time: ~50 us

First conversion after init should be discarded for best accuracy

Example:

```
rx_err_t err = rx_bus_adc_init(&manager, "motor_current");
if (err != k_rx_ok) {
    rx_log_error("APP", "Failed to init ADC: %d", err);
}
```

See also

rx_bus_adc.h Complete API documentation

internal_adc_init_callback() Implementation callback

Since

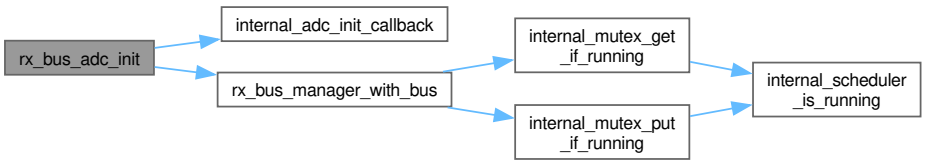
Version 1.0.0

Definition at line 674 of file rx_bus_adc.c.

```
00675 {
00676   RX_CHECK_NULL_PTR(manager, s_tag, "manager pointer is nullptr");
00677   RX_CHECK_NULL_PTR(bus_name, s_tag, "bus_name pointer is nullptr");
00678
00679   adc_init_ctx_t ctx = {.result = k_rx_err_hw_error};
00680
00681   const rx_err_t err = rx_bus_manager_with_bus(manager, bus_name, internal_adc_init_callback, &ctx);
00682
00683   if (err != k_rx_ok) {
00684     return err;
00685   }
00686
00687   return ctx.result;
00688 }
```

References internal_adc_init_callback(), adc_init_ctx_t::result, rx_bus_manager_with_bus(), and s_tag.

Here is the call graph for this function:



7.1.2.2 rx_bus_adc_read()

```
rx_err_t rx_bus_adc_read (
                rx_bus_manager_t * manager,
                const char * bus_name,
                uint16_t * value)    [nodiscard]
```

Read ADC raw value through bus manager.

Performs a single ADC conversion and returns the raw digital value (0-4095 for 12-bit). Does not perform voltage scaling - use rx_bus_adc_read_voltage_mv() for automatic voltage conversion.

Algorithm Steps:

1. Validate all pointers (manager, bus_name, value)
2. Acquire mutex with timeout
3. Lookup bus configuration
4. Validate bus initialized
5. Start ADC conversion (software trigger)
6. Wait for conversion complete (poll ADCSR.ADST bit)
7. Read result from ADDRn register
8. Release mutex
9. Return raw value

Conversion Timing:

```
* Timeline for single ADC read:
*
* |<--- Sampling --->|<- Conversion ->|
* |     3.0 us       |     1.5 us     |
* |                                   |
* Start                          Result Ready
*
* Total: 4.5 us typical @ 12-bit resolution
*
```

Parameters

in	manager	Bus manager instance • Must be initialized
in	bus_name	ADC bus name • Must match registered bus
out	value	Pointer to store raw ADC value • Range: 0-4095 (12-bit resolution) • 0 = 0V input, 4095 = VREF input • Valid only if k_rx_ok returned

Returns

rx_err_t Error code indicating result

Return values

k_rx_ok	Success, value contains raw ADC reading
k_rx_err_null_ptr	nullptr in any parameter
k_rx_err_not_found	Bus name not found
k_rx_err_invalid_state	Bus not initialized
k_rx_err_timeout	Conversion timeout or mutex timeout

Precondition

Bus must be initialized via rx_bus_adc_init()

value pointer must be valid

Analog input voltage must be within 0 to VREFH

Postcondition

value contains raw 12-bit ADC result

ADC ready for next conversion

Invariant

Bus state remains initialized

value range: 0-4095 for 12-bit ADC

Note

Thread-safe via bus manager mutex

Blocking call (waits for conversion ~4.5 us)

For voltage conversion, use rx_bus_adc_read_voltage_mv() instead

Warning

value undefined if return != k_rx_ok

Analog input must not exceed VREFH (damage possible)

Performance:

- Execution time: ~10 us (4.5 us conversion + overhead)
- Conversion timeout: 100 us (configurable)

Example - Raw ADC Reading:

```
uint16_t raw_adc = 0;
rx_err_t err = rx_bus_adc_read(&manager, "sensor_input", &raw_adc);
if (err == k_rx_ok) {
    // Convert to voltage manually
    uint32_t voltage_mv = (raw_adc * 3300UL) / 4095;

    // Or use in lookup table
    float temperature_c = ntc_thermistor_table[raw_adc];
}
```

Example - Multiple Readings for Averaging:

```
// Average 16 readings for noise reduction
uint32_t sum = 0;
for (uint8_t i = 0; i < 16; i++) {
    uint16_t reading = 0;
    err = rx_bus_adc_read(&manager, "noisy_signal", &reading);
    if (err == k_rx_ok) {
        sum += reading;
    }
}
uint16_t average = sum / 16;
```

See also

rx_bus_adc_init() Initialize ADC first

rx_bus_adc_read_voltage_mv() Read with automatic voltage conversion

Since

Version 1.0.0

Read ADC raw value through bus manager.

Public API function to perform single ADC conversion and return raw digital value (0-4095 for 12-bit). No voltage scaling is performed - use rx_bus_adc_read_voltage_mv() for automatic voltage conversion. Delegates to bus manager which calls internal_adc_read_callback() with mutex protection.

Implementation pattern:

1. Validate all pointers (manager, bus_name, value)
2. Create stack-allocated context with value pointer
3. Call rx_bus_manager_with_bus() to execute callback
4. Extract result from context (value already updated by callback)
5. Return result to caller

Precondition

Bus must be initialized via rx_bus_adc_init()

All pointers must be non-NULL

value must point to valid uint16_t

Postcondition

*value contains raw 12-bit ADC result (if k_rx_ok)

ADC ready for next conversion

Note

Thread-safe via bus manager mutex

Blocking call - waits for conversion (~4.5 us)

Stack usage: 16 bytes for context

Execution time: ~10 us

Warning

value undefined if return != k_rx_ok

Analog input must not exceed VREFH

Example:

```
uint16_t raw_adc = 0;
rx_err_t err = rx_bus_adc_read(&manager, "sensor", &raw_adc);
if (err == k_rx_ok) {
    // Convert to voltage manually
    uint32_t voltage_mv = (raw_adc * 3300UL) / 4095;
}
```

See also

rx_bus_adc.h Complete API documentation

internal_adc_read_callback() Implementation callback

rx_bus_adc_read_voltage_mv() Read with automatic voltage conversion

Since

Version 1.0.0

Definition at line 739 of file rx_bus_adc.c.

```
00740 {
00741   RX_CHECK_NULL_PTR(manager, s_tag, "manager pointer is nullptr");
00742   RX_CHECK_NULL_PTR(bus_name, s_tag, "bus_name pointer is nullptr");
00743   RX_CHECK_NULL_PTR(value, s_tag, "value pointer is nullptr");
00744
00745   adc_read_ctx_t ctx = {.value = value, .result = k_rx_err_hw_error};
00746
00747   const rx_err_t err = rx_bus_manager_with_bus(manager, bus_name, internal_adc_read_callback, &ctx);
00748
00749   if (err != k_rx_ok) {
00750     return err;
00751   }
00752
00753   return ctx.result;
00754 }
```

References internal_adc_read_callback(), adc_read_ctx_t::result, rx_bus_manager_with_bus(), and s_tag.

Here is the call graph for this function: